The AI

Fellowship

LLM Engineering Workshop

Prompt Engineering

Advanced Patterns — Session 2, Part B

Aryan | Robotics Lead @Hexaware

Agenda

01 **ReAct Pattern**

Reasoning + Acting + Observing · Thought→Action→Observation cycle · When to use vs CoT

02 **Self-Consistency & Multiple Sampling**

Sample-and-vote strategy · Optimal N · Weighted vs majority voting · When NOT to use it

03 **Prompt Chaining & Task Decomposition**

Three chain architectures · Context passing without bloat · When to chain vs single prompt

04 **Role-Based Prompting & Personas**

What research says about roles · Designing effective roles · Three categories that work

05 **Meta-Prompting & Prompt Optimization**

APE · DSPy · OPRO · Manual + LLM assist · When automated optimization pays off

01

REACT PATTERN

Reasoning · Acting · Observing · Grounded multi-step problem solving

Thought → Action → Observation

Tool grounding

0% hallucination

Yao et al. 2022

Three Numbers That Define Advanced Prompting

0%

ReAct Hallucination Rate on FEVER

CoT hallucinated in **56% of failure cases** on the FEVER fact-verification benchmark. ReAct hallucinated in none — because every next thought is grounded in a real tool observation, not a model's memory.

Rule: When facts matter, ground the model in retrieved evidence — not its parametric memory.

— Yao et al., ReAct (2022)

71%

ReAct Success Rate on ALFWorld

ReAct achieved **71% task success** on ALFWorld interactive tasks vs **45% for action-only prompting** — a 26-point jump from simply adding a reasoning trace before each action.

Rule: Thinking before acting isn't just good advice — it's measurably better prompting.

— Yao et al., ReAct (2022)

33% →82%

DSPy on GSM8K: GPT-3.5 Before and After

DSPy compiled prompt optimization raised GPT-3.5 accuracy on GSM8K from **33% to 82%** — a 49-point lift by treating prompts as **learnable parameters** rather than hand-written strings.

Rule: High call volume + stable task + clear eval metric = automate your prompt optimization.

— Khattab et al., DSPy (2023)

The ReAct Cycle — Reasoning Grounded in Reality

1 Thought

The model reasons about what it needs to do next — before acting. Thinking is explicit, not hidden.

```
Thought: I need to find when the Eiffel Tower was built.
```

↓

2 Action

The model calls a tool with a precise, scoped query.

```
Action: web_search["Eiffel Tower construction date"]
```

↓

3 Observation

The tool result is appended to context. The model reads it and forms its next Thought — grounded in real data.

```
Observation: The Eiffel Tower was constructed 1887–1889...
```

↓

4 Finish

One of three termination conditions fires: goal achieved (finish), max steps reached, or unrecoverable error.

```
Action: finish["The Eiffel Tower opened in 1889."]
```

Why ReAct Wins

Benchmark comparison (Yao et al. 2022, PaLM-540B)

Benchmark	CoT	ReAct
FEVER hallucination	56%	**0%**
ALFWorld success	—	**71%**
HotpotQA (EM)	29.4	27.4 → 35.1*

** Best combo: ReAct + CoT Self-Consistency*

PROMPT ENGINEERING RULES

🏷 Tool descriptions must be unambiguous — if a human can't choose the right tool, neither can the model.

🔢 Use 3–6 few-shot examples. More didn't improve results in the original paper.

⛔ Always enforce a max step limit. <1.4% of correct trajectories hit the cap.

Three Prompt Chain Architectures

Each link in a chain gets the model's full attention on one clear objective — that's why chains beat monolithic prompts on complex tasks

① Sequential

`A → B → C`

Each step's output feeds directly into the next. Each link gets full model attention on one clear objective.

BEST FOR

Research → Extract → Summarize pipelines

> ⚡ *Pass only extracted data between steps — not full prose. Context explosion is the #1 chaining pitfall.*

USE WHEN

- Output quality drops on complex instructions
- You need intermediate validation
- Steps must happen in order

② Branching

`Classify → Route A / B / C`

Classify the input first, then route to a specialised prompt per class. Enables different models or temperatures per branch.

BEST FOR

Customer support routing · Content moderation · Multi-format processing

> ⚡ *The classifier must be highly reliable — misroutes cascade silently through the whole pipeline.*

USE WHEN

- Different inputs need different handling
- Steps are independent per class
- You want different models per branch

③ Iterative

`Draft → Critique → Revise → (repeat)`

Generate, critique, and revise in a loop until a quality threshold is met. Requires a clear stopping criterion.

BEST FOR

Writing · Code review · Any task with subjective quality

> ⚡ *Define your termination condition upfront. Without it, iteration loops indefinitely and costs multiply.*

USE WHEN

- Output has subjective quality criteria
- First draft is rarely production-ready
- You can evaluate output programmatically

Context tip: Use XML tags (<step1_output>…</step1_output>) to delineate chain steps — Anthropic recommendation

Advanced Reasoning Frameworks

Cost, latency, and best task type — pick the right tool for the job

S

+17.9pp on GSM8K

Self-Consistency

Cost: 5–15× · Low (parallel)

BEST FOR

Math · Classification · Verification

LIMITATION

No help for creative or open-ended tasks

N=5 captures 70–80% of max gain. Simple majority vote — skip confidence weighting, it adds nothing.

R

71% ALFWorld SR

ReAct

Cost: 3–10× · High (sequential)

BEST FOR

Tool use · Dynamic exploration · Fact grounding

LIMITATION

Loop risk · Context grows quadratically per step

Each step adds 0.5–2s. 23% of ReAct failures in the original paper came from uninformative search results.

T

74% Game of 24

Tree of Thoughts

Cost: 5–100× · Very high

BEST FOR

Search & planning with dead ends · Puzzles

LIMITATION

Exponential cost · Complex setup

CoT achieved 4% on Game of 24. ToT reached 74%. Cost: ~5,500 completion tokens per problem.

X

91% HumanEval

Reflexion

Cost: 3–9× · Very high

BEST FOR

Code generation · Tasks with clear pass/fail

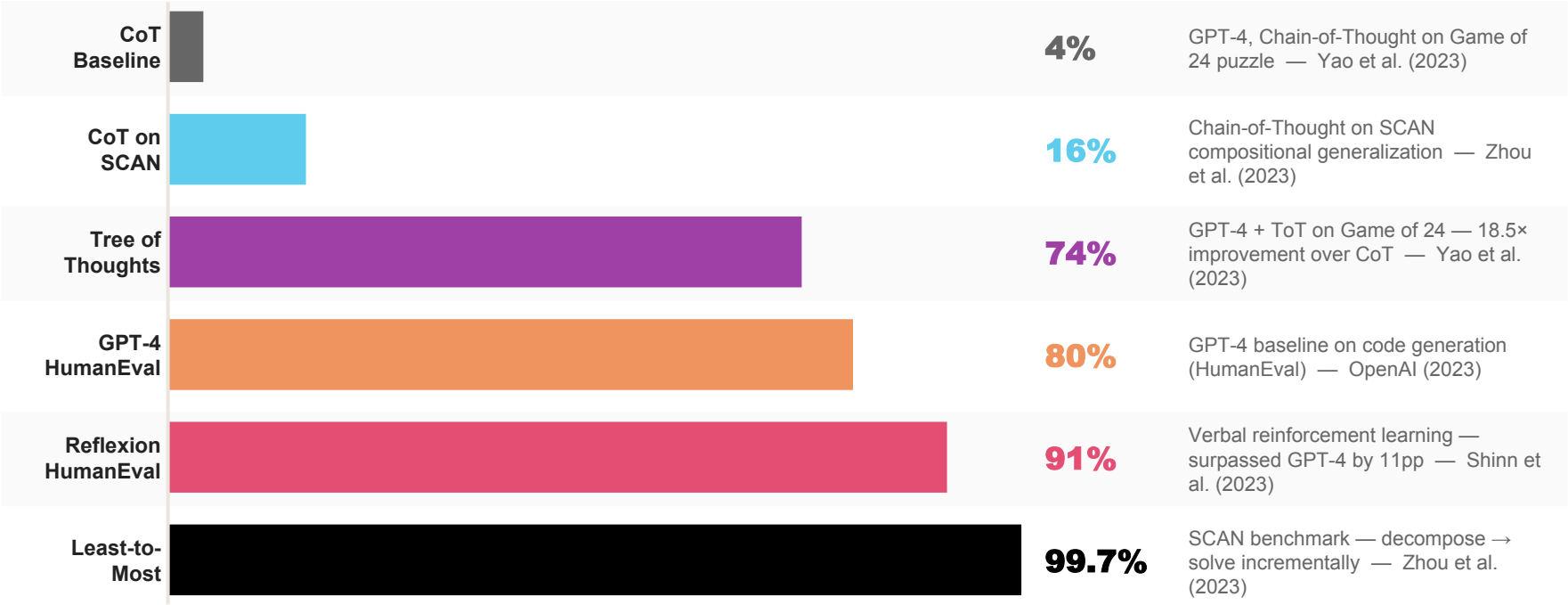
LIMITATION

Needs objective evaluation signal to reflect on

Surpassed GPT-4's then-SOTA of 80% on HumanEval. Sequential & iterative — learns from each failure.

Accuracy Milestones Across Advanced Frameworks

Each breakthrough came from a different architectural insight — not just a bigger model



The pattern: each framework solved a specific failure mode of the one before it. CoT can't backtrack → ToT. CoT/ReAct can't learn from failure → Reflexion. CoT can't generalize compositionally → Least-to-Most.

Advanced Reasoning Frameworks at a Glance

Start simple — add complexity only when measurement shows a gap

Framework	Cost	Latency	Best Task Type	Key Limitation
CoT	1×	Lowest	Simple reasoning, math	Hallucination on knowledge tasks (56% on FEVER)
Self-Consistency	5–15×	Low (parallel)	Math · Classification · Verification	No help for creative or open-ended tasks
ReAct	3–10×	High (sequential)	Tool use · Dynamic exploration	Loop risk · Context bloat per step
Prompt Chaining	2–5×	Medium	Multi-step pipelines	Error propagation across steps
Tree of Thoughts	5–100×	Very high	Search / planning with dead ends	Exponential cost · Complex setup
Reflexion	3–9×	Very high	Code · Tasks with clear pass/fail	Needs objective evaluation signal
Constitutional AI	2–3×	Medium	Safety · Tone · Policy compliance	Extra LLM call for every output
Least-to-Most	2–5×	High (sequential)	Compositional / hierarchical tasks	Decomposition quality is the bottleneck

Rule of thumb: CoT first. Need tools → ReAct. Higher accuracy → Self-Consistency. Alternatives needed → Tree of Thoughts. Safety guardrails → Constitutional AI.

Start simple. Most tasks only need direct prompting or basic CoT. Add complexity only when measurement shows a gap.

Two Non-Obvious Research Insights

01 Simple majority voting matches weighted voting for self-consistency. The model can't reliably judge its own confidence — skip log-probability weighting.

02 Persona prompting rarely improves factual accuracy despite its popularity. Use roles for behavioral shaping and tone control — not knowledge activation.

Ineffective vs Effective Role Prompting

Roles work for style, tone, and format — not factual accuracy. Specify behaviour, not just identity.

✕ Useless Role

```
"You are a doctor."
```

Why this fails:

- ✕ Specifies identity only — no change in reasoning behaviour
- ✕ 162 personas tested across 4 LLM families found no or small negative effects on accuracy vs no-persona baseline
- ✕ GPT-4-turbo: system role prompts didn't improve performance and sometimes hurt (Learn Prompting, 2023)
- ✕ Model defaults to generic doctor-like phrasing — not clinical specificity

vs

✓ Effective Role

```
"You are a board-certified cardiologist
reviewing patient cases. You always
list differential diagnoses ranked by
likelihood and flag any red-flag
symptoms requiring immediate attention."
```

Why this works:

- **Functional** — Specifies exact output behaviour (ranked list)
- **Scoped** — Defines the action ('flag red-flag symptoms')
- **Measurable** — Output is verifiable — you know what to expect

3 categories that work: Expert framing · Functional roles · Audience roles

⚠ Common mistakes: over-specifying irrelevant details (persona names, backstories) can degrade performance by up to 30pp · Conflicting traits ('thorough but extremely brief') · Fictional personas activate creative writing patterns

Meta-Prompting & Prompt Optimization Toolkit

Using LLMs to improve LLMs' instructions — from manual iteration to compiled optimization

APE

Automatic Prompt Engineer — Zhou et al. (2022)

Generate candidate prompts → score on eval set → select best

→

Matched or exceeded human-written prompts on 24/24 instruction induction tasks. Discovered a CoT trigger that outperformed "Let's think step by step."

Discovering novel instructions from scratch

DSPy

Declarative Self-improving Python — Khattab et al. (2023)

Declarative input/output signatures + compiled optimization. Treats prompts as learnable parameters.

→

GPT-3.5 accuracy on GSM8K: 33% → 82% — a 49-point lift. dspy.ai

Multi-stage pipelines · Systematic optimization at scale

OPRO

Optimization by PROmpting — Yang et al. (2023)

LLM-as-optimizer: feed current prompt + failure examples → LLM produces improved version iteratively

→

Effective for refining prompts that are already working but need marginal improvement

Iterative refinement of existing production prompts

Manual + LLM Assist

Human writes · LLM critiques and rewrites

Human writes the initial prompt. LLM critiques, identifies weak spots, and rewrites. No training data required.

→

Fastest path for exploratory or rapidly changing tasks where automated methods are overkill

Rapid prototyping · Low-data scenarios · Early-stage tasks

When Automated Optimization (DSPy / APE) Pays Off:

- High call volume (>10K calls/day on the same prompt)
- Stable task definition with clear, measurable eval metrics
- Training examples available to score against

AI for every human.

Session 2, Part B — Advanced Prompt Engineering Patterns

www.theaifellowship.com